Universidad Nacional Autónoma de México
Facultad de Ciencias

Computación Distribuida 2027-1

Práctica 01

Introducción a la Computación Distribuida

Profesores:
Fernando Michel Tavera
Luis Mario Escobar Rosales
David Ortega Medina

Alumnos:

- Jesús Echeverría Verduzco **No. Cuenta:** 321239167
- José Carlos Pinzón Chan **No. Cuenta:** 425122448

Durante está práctica se implementaron algunos algoritmos básicos relacionados con la computación distribuida. Específicamente, se trabajó con algoritmos asociados al paso de mensajes entre distintos nodos de un sistema distribuido:

Algoritmo 1 (NodoVecinos): Permite a un nodo del sistema conocer información sobre sus vecinos. En este caso, permite conocer de cierto modo el "alcance" que tiene este nodo, osea, los nodos que desde él pueden ser alcanzados (existe un canal de comunicación, puede o no ser directo). De esta manera se proporciona algo de información sobre la topología de la red.

Para poder llevar a cabo la realización del algoritmo de NodosVecinos se completaron las siguientes funciones:
– static void recibir myname(Nodo *nodo, const Mensaje *mensaje,Simulador *simulador)
– bool nodo vecinos iniciar(Nodo *nodo, Simulador*simulador)

La primera función es bastante sencilla. Inicialmente revisa que el tipo del mensaje sea MY_NAME, de esta manera sabe que el algoritmo que se lleva a cabo es el de NodoVecinos. Después, el mensaje trae consigo un conjunto de identificadores (enteros), así que simplemente agregamos los identificadores de los vecinos del nodo que recibe el mensaje, al conjunto.

Por su parte, la segunda función, es la que crea el mensaje y simula la propagación de solicitudes para recopilar la información.

Algoritmo 2 (NodoTopología): Como el nombre sugiere, este algoritmo nos permite conocer la topología de la red a la que pertenece un nodo. Dependiendo de la topología REAL de la red, esta función a veces puede describir sólo parcialmente la topología de la red REAL (por ejemplo si la topología REAL de la red es una topología no conexa).

¿Cómo se describe la topología? Existe un estado de la topología que agrupa varios datos sobre ella: La cantidad de nodos, los nodos conocidos, los canales de comunicación conocidos y si es completa.

Para poder llevar a cabo la realización del algoritmo de NodosTopología se completaron las siguientes funciones:

– static bool verificar_completa(const EstadoTopologia *estado)
– static void recibir position(Nodo *nodo,const Mensaje *mensaje,Simulador *simulador)
– bool nodo topologia iniciar(Nodo *nodo, Simulador *simulador)

La función verificar_completa parte de la pregunta de *si la topología del sistema distribuido es completa.* Para responder esta pregunta, revisamos que para todas las posibles combinaciones de tamaño 2 de nodos, por ejemplo el nodo 1 y 2 o 4 y 5, existe el índice del canal en los canales conocidos por la topología. Similarmente, se verifica que cada proceso (nodo) de la combinación existe en los procesos conocidos por la topología. Si esto no ocurriera con alguna de las combinaciones, entonces la topología no sería completa (haría falta al menos un canal).

Si ninguna de las verificaciones falla, se regresa true indicando que la topología es completa.

Luego, la segunda función registra todos los canales de un nodo, dicho nodo es previamente marcado como registrado y después trata de distribuir un mensaje actualizado a los demás nodos.

Por último la tercera función simplemente simula el envío del mensaje MSG_POSITION desde un nodo destacado en alguna topología. Esencialmente esta es la que realmente ejecuta el algoritmo completo.

Algoritmo 3 (NodoBroadCast): Se tiene un nodo designado que será el emisor de un mensaje. Este algoritmo envía el mensaje a todos los vecinos del nodo designado, los vecinos marcan como visto el mensaje y lo propagan con sus propios vecinos (esto incluye al nodo emisor). Cuando un nodo recibe el mensaje por primera vez, lo marca como visto. Este procedimiento sigue hasta que todos los nodos hayan visto el mensaje y por tanto en la última propagación ninguno reaccione (se detiene la propagación).

La implementación de este tipo de broadcast se dice que es “ingenua” porque no existe ningún método para intentar reducir las redundancias (enviar el mensaje a nodos que ya lo han recibido).

– static void recibir broadcast(Nodo *nodo, const Mensaje *mensaje, Simulador *simulador)
– bool nodo broadcast iniciar(Nodo *nodo, Simulador *simulador)

El primer método se encarga de procesar los mensajes de tipo Broadcast recibidos por un nodo, un nodo es capaz de recibir información difundida por otro nodo del sistema y utilizarla para actualizar la información de los nodos que actualmente se conocen sobre la red.

A Través de múltiples verificaciones, finalmente es capaz de atender los mensajes recibidos y realizar las acciones necesarias para que la información pueda propagarse a través del sistema distribuido.

Por otro lado para realizar la propagación de mensaje a través del segundo método, se necesita saber si el mensaje proviene a partir de un nodo fuente, que después se capaz de comparar si tiene el correcto tipo de mensaje y a partir de ello, realizar la réplica del mensaje a través de un mensaje local que le transmite la orden a sus vecinos, finalmente el método elimina cualquier rastro total en caso de error para disponer de un nuevo orden de mensaje a procesar y en caso de ser visitado el sistema es capaz de evitarlo hasta recorrer todo el sistema distribuido.